CHAPITRE 4

Variables locales/globales

PTSI

2026 – 2027

PLAN DU CHAPITRE

4.1 Passage d'un argument à une fonction 2

4.2 Variable locale, variable globale 2

4.3 Cas particulier des listes 4

4.1 Passage d'un argument à une fonction

Une variable utilisée dans le corps d'une fonction est créée au moment de l'appel de cette fonction, et détruite à sa sortie. Lorsqu'on transmet un entier, un flottant, une chaîne ou un booléen en argument, tout se passe comme si la fonction en recevait une **copie** : la modifier à l'intérieur de la fonction n'a aucune incidence sur la variable d'origine.

incremente.py

```
1 def incremente(x):
2     x = x + 1
3     return x
```

console

```
In [1]: x = 1
In [2]: incremente(x)
Out[2]: 2
In [3]: x
Out[3]: 1
```

4.2 Variable locale, variable globale

Définition 4.1 · Portée d'une variable

Une variable est dite *globale* si elle est définie dans l'environnement principal du programme (en dehors de toute fonction); elle est dite *locale* si elle est définie à l'intérieur du corps d'une fonction. Une variable locale est créée à l'appel de la fonction qui la définit, et détruite dès la sortie de cette fonction. Toute tentative d'y accéder depuis l'extérieur produit une erreur.

console

```
In [1]: def fonct():
   ...:     y = 1
   ...:
In [2]: fonct()
In [3]: y
NameError: name 'y' is not defined
```

Une fonction peut lire une variable globale, mais pas la modifier directement

À l'intérieur d'une fonction, on peut **consulter** la valeur d'une variable globale sans problème. En revanche, dès qu'on tente de lui **affecter** une nouvelle valeur, Python la considère comme une variable locale à part entière, ce qui produit une erreur si on essaie de l'utiliser avant de l'avoir définie localement :

console

```
In [1]: x = 10
In [2]: def fonct():
   ...:     print(x)
   ...:     x = x + 1
   ...:
In [3]: fonct()
10
UnboundLocalError: local variable 'x' referenced before assignment
```

Définition 4.2 · Identifiant d'un objet

La fonction `id(objet)` renvoie un entier identifiant de façon unique l'emplacement mémoire d'un objet. Deux variables désignant le même objet partagent le même identifiant ; deux objets distincts, même de valeur égale, ont des identifiants différents.

Si l'on crée, à l'intérieur d'une fonction, une variable locale portant le **même nom** qu'une variable globale déjà existante, il s'agit en réalité de deux variables totalement distinctes (on peut s'en convaincre en comparant leurs identifiants avec `id`) :

console

```
In [1]: x = 10
In [2]: id(x)
Out[2]: 263937988739600

In [3]: def fonct():
   ...:     x = 1
   ...:     print(id(x))
   ...:     x = x + 1
   ...:     print(x)
   ...:
In [4]: fonct()
263937988739312
2
```

Remarque

On évite autant que possible de donner le même nom à une variable locale et à une variable globale, la confusion étant vite arrivée.

4.3 Cas particulier des listes

Une liste transmise en argument à une fonction ne se comporte **pas** comme un entier ou une chaîne : c'est une référence vers le même objet en mémoire qui est transmise. Certaines opérations modifient donc directement la liste de l'appelant, sans qu'il soit besoin d'un `return` ni d'un `global` :

- ▷ les **méthodes** qui modifient la liste en place (`append`, `insert`, `pop`, `remove`, `sort`, ...), la commande `del`, ou la modification directe d'un élément (`L[i] = ...`) agissent **globalement** ;
- ▷ en revanche, réaffecter le nom `L` lui-même (par exemple avec `L = L + [...]`, ou `L = [...]`) ne crée qu'une nouvelle liste **locale** : la liste de l'appelant reste inchangée.

console

```
In [1]: def vide(L):
   ...:     L = [0]*len(L)    # une NOUVELLE liste est creee ici
   ...:     return L
   ...:
In [2]: L = [1, 2]
In [3]: vide(L)
Out[3]: [0, 0]
In [4]: L
Out[4]: [1, 2]

In [5]: def rallonge(L):
   ...:     L.append(0)       # modification EN PLACE de la liste recue
   ...:     return L
   ...:
In [6]: L = [1, 2]
In [7]: rallonge(L)
Out[7]: [1, 2, 0]
In [8]: L
Out[8]: [1, 2, 0]
```

Copie ou alias ?

Pour un objet de type `int`, `float`, `str` ou `bool`, l'instruction `b = a` recopie effectivement la valeur de `a` dans `b` : modifier ensuite `a` n'affecte pas `b`. Il n'en va pas de même pour un conteneur (liste, tableau, dictionnaire), `L2 = L` ne crée pas de copie, mais un *alias* : `L2` devient simplement un second nom désignant le même emplacement mémoire que `L`. Pour obtenir une véritable copie, il faut utiliser `L2 = L.copy()`.

console

```
In [1]: a = 4
In [2]: b = a
In [3]: a = 6
In [4]: b
Out[4]: 4

In [5]: LA = [1,3,5]
In [6]: LB = LA           # alias
In [7]: LA.append(7)
In [8]: LB
Out[8]: [1, 3, 5, 7]

In [9]: LA = [1,3,5]
In [10]: LB = LA.copy()    # copie
In [11]: LA.append(7)
In [12]: LB
Out[12]: [1, 3, 5]
```

On peut d'ailleurs coder soi-même l'équivalent de `.copy()`, en recopiant terme à terme à l'aide d'une boucle `for` :

copie.py

```
1 def copie(L):
2     Ls = []
3     for elem in L:
4         Ls.append(elem)
5     return Ls
```

Bonnes pratiques : deux façons d'écrire une fonction qui modifie une liste

Face à une fonction censée modifier une liste, deux approches sont possibles :

- **de façon procédurale** : on modifie `L` directement, *en place*, sans `return`. On appelle alors la fonction sous la forme `f(Ltst)` (et surtout pas `Ltst = f(Ltst)`, puisque la fonction ne renvoie rien);
- **en travaillant sur une copie** : on commence par dupliquer `L` avec `.copy()`, on ne modifie ensuite que cette copie (jamais `L`), et on la renvoie à la fin avec `return`.

change_1_terme.py

```python
# Version procedurale (en place)
def change_1_terme(L):
    L[0] = "zero"

# Version avec copie
def change_1_terme_copie(L):
    Lc = L.copy()
    Lc[0] = "zero"
    return Lc
```

console

```
In [1]: Ltst = [1,3,5]
In [2]: change_1_terme(Ltst)
In [3]: Ltst
Out[3]: ['zero', 3, 5]

In [4]: Ltst = [1,3,5]
In [5]: change_1_terme_copie(Ltst)
Out[5]: ['zero', 3, 5]
In [6]: Ltst
Out[6]: [1, 3, 5]
```

Tant que la distinction entre ces deux approches n'est pas parfaitement maîtrisée, il est recommandé de privilégier la version **avec copie**.